



**UNIVERSIDAD  
NACIONAL DE  
INGENIERÍA**



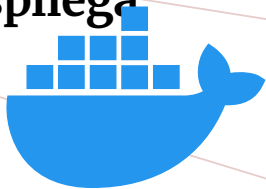
TRANSFORMACIÓN  
**digital**

**CURSOS GRATUITOS**

**OTI UNI**

# Docker desde Cero: Crea y Despliega Aplicaciones

Programa Completo — PIT 2026



---

Instructor:  **Anthony Gonzalo Quispe Cordova**

Oficina de Tecnologías de la Información (OTI)

9na Edición

# Índice General del Programa

---

**1** S1 · Introducción y Fundamentos

**2** S1 · Virtualización e Hipervisores

**3** S1 · Instalación de Docker

**4** S1 · Imágenes y Contenedores

**5** S1 · Comandos Esenciales

**6** S1 · Primera Aplicación

**7** S2 · Recordatorio

**8** S2 · Instrucciones Clave del Dockerfile

**9** S2 · Capas, Caché y Buenas Prácticas

**10** S2 · .dockerignore e Imágenes Base Livianas

**11** S2 · Publicación en Docker Hub

**12** S2 · Laboratorio Práctico

**13** S3 · Docker Compose

**14** S3 · Estructura de docker-compose.yml

**15** S3 · Flask con PostgreSQL

**16** S3 · Variables con .env

**17** S3 · Laboratorio Compose

**18** S4 · Redes Docker

**19** S4 · Volúmenes y Persistencia

**20** S4 · Aislamiento de Servicios

**21** S4 · Healthchecks y Backup

**22** S4 · Laboratorio Datos y Redes

# Objetivo de la sesión 1

---

## Al terminar la sesión 1 podrás

- Explicar el concepto de contenerización.
- Explicar qué problema resuelve Docker.
- Ubicar Docker frente a VM e hipervisores.
- Identificar el flujo para instalar una VM base.
- Saber dónde instalar Docker y Compose desde la documentación oficial.
- Diferenciar imagen y contenedor.
- Ejecutar comandos básicos: `run`, `ps`, `stop`, `rm`.
- Construir y ejecutar una primera app en contenedor.



**Menos teoría, más terminal**

---

# Introducción y fundamentos

# El problema: «funciona en mi máquina»

---

- La app funciona en la laptop del desarrollador.
- En otro equipo falla por versiones distintas.
- En producción aparecen librerías faltantes o configuraciones diferentes.
- El equipo pierde tiempo reconstruyendo el entorno.

## Clave

Docker busca que el entorno viaje junto con la aplicación.



**«En mi PC funciona»**

# Solución: la contenerización

---

- Aislar la aplicación en un **contenedor** reproducible.
- Empaquetar código, dependencias y configuración en una **imagen**.
- Compartir esa imagen con el equipo o con un registro central.
- Ejecutarla en distintos entornos con menos diferencias.

## Resultado

La app deja de depender de «lo que justo estaba instalado» en una máquina.



**App + dependencias**

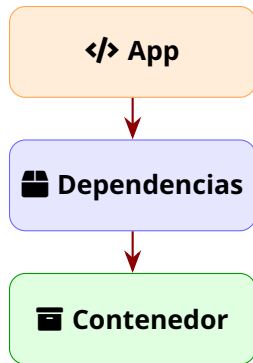
# Qué es la contenerización

---

## En palabras simples

La contenerización empaqueta una aplicación con lo necesario para ejecutarla de forma uniforme en diferentes entornos.

- Aplicación + dependencias.
- Configuración mínima reproducible.
- Ejecución aislada del resto del sistema.
- Menos problemas de compatibilidad.



# La pregunta natural

---

## Ya tenemos la idea

Contenerizar es empaquetar una app con sus dependencias para ejecutarla igual en distintos entornos.

## Ahora falta la herramienta

**Docker** es una de las formas más usadas para construir, compartir y ejecutar esos contenedores.

Entonces... ¿qué es Docker?



**Docker =**

Construir · Compartir · Ejecutar  
*contenedores*



Entonces... ¿qué es Docker?

---

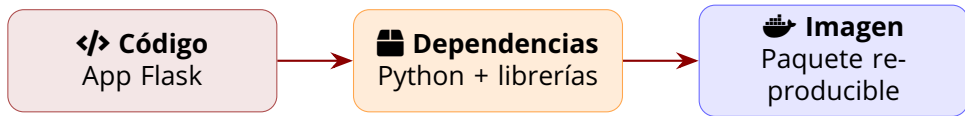
**Si una VM virtualiza una  
máquina...**

**Docker ejecuta  
aplicaciones  
en contenedores**

---

# Docker en una frase

---



## Definición práctica

Docker permite empaquetar una aplicación con sus dependencias para ejecutarla de forma consistente en diferentes equipos.

# Docker: introducción práctica

---

- Docker es una plataforma *open source* para crear, compartir y ejecutar contenedores.
- Implementa la idea de contenerización en el flujo diario de desarrollo.
- Facilita pruebas, despliegues y trabajo en equipo.
- Se apoya en imágenes, contenedores, registros y la CLI de Docker.

## Idea central

Docker no reemplaza toda VM: resuelve la ejecución reproducible de aplicaciones.



**Docker**

# Qué sí veremos y qué no veremos hoy

---

## Hoy sí

- Concepto de Docker.
- Concepto de contenerización.
- Ruta oficial de instalación.
- Imagen vs contenedor.
- Comandos esenciales.
- Primera app con Dockerfile.

## Hoy no

- Kubernetes, OCI profundo o cgroups.
- YAML avanzado.
- Redes y volúmenes a detalle.
- Producción con Nginx.

Pero... ¿cómo funciona?

---

**Entendido... ¿Pero cómo funciona?**

*¿Es Docker simplemente otra...*

**Máquina Virtual?**

---

---

# Virtualización e hipervisores

# Antes de Docker: máquinas virtuales

---

- Una VM ejecuta un sistema operativo completo sobre hardware virtual.
- Permite aislar entornos: Linux, Windows Server, laboratorios, pruebas.
- Es útil cuando necesitas simular una máquina completa.
- Consume más recursos que un contenedor porque incluye su propio kernel.

## Idea clave

La VM virtualiza una máquina; Docker empaqueta procesos aislados.



**Máquina completa**

# Qué es un hipervisor

---

## Tipo 1

Corre directo sobre el hardware. Ejemplos: VMware ESXi, Hyper-V Server, Proxmox.

## Tipo 2

Corre como aplicación del sistema host. Ejemplos: VirtualBox, VMware Workstation.

## Para este curso

Si necesitas un laboratorio local, una VM con Linux puede servir como entorno limpio para instalar Docker y practicar sin afectar tu sistema principal.



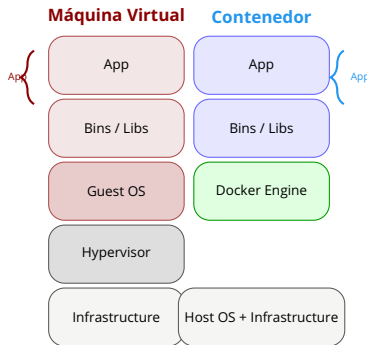
**¿Y Docker?**

*¿Es un hipervisor?*

*¿De qué tipo?*

---

# Docker vs máquina virtual



## Diferencia clave

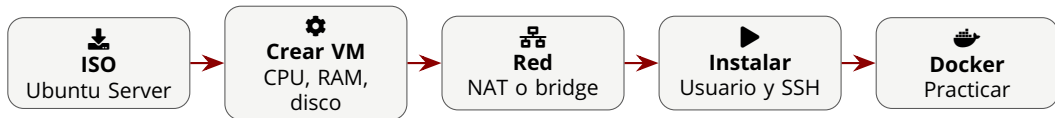
La VM incluye un sistema operativo completo (Guest OS). El contenedor comparte el kernel del host y solo incluye lo que la app necesita.

# VM vs contenedor: decisión rápida

---

| Criterio          | Máquina virtual           | Contenedor               |
|-------------------|---------------------------|--------------------------|
| Sistema operativo | Incluye SO completo       | Comparte kernel del host |
| Arranque          | Más lento                 | Rápido                   |
| Consumo           | Mayor RAM y disco         | Menor consumo            |
| Aislamiento       | Fuerte a nivel máquina    | Aislado a nivel proceso  |
| Uso común         | Servidores y laboratorios | Apps reproducibles       |

# Instalando una VM: flujo recomendado



## Configuración sugerida

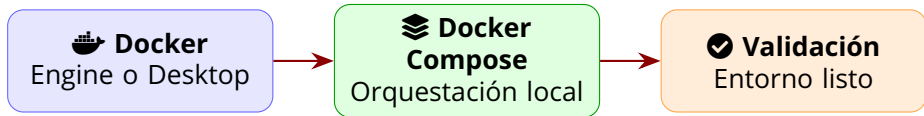
2 CPU, 4 GB RAM, 25 GB disco, red NAT para empezar. Si necesitas acceso desde otros equipos, usa bridge.

---

# Instalación de Docker

# Qué debes instalar

---



## Criterio del curso

No memorices comandos de instalación: pueden cambiar por sistema operativo. Usa siempre la guía oficial y valida según tu plataforma.

# Ruta oficial según tu sistema

---

## Windows y macOS

- Instalar **Docker Desktop**.
- Incluye Docker Engine, CLI y Docker Compose.
- Revisar requisitos de WSL 2 en Windows.

 [docs.docker.com/desktop](https://docs.docker.com/desktop)

## Linux o VM Linux

- Instalar **Docker Engine** según distribución.
- Revisar pasos post-instalación.
- Instalar Compose desde la guía oficial.

 [docs.docker.com/engine/install](https://docs.docker.com/engine/install)

# Docker Compose y enlaces oficiales

| Tema                                    | Cuándo usarlo                                                       | Documentación                                                       |
|-----------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------|
| Docker Desktop                          | Windows/macOS o experiencia integrada                               | <a href="#">Docker Desktop</a>                                      |
| Docker Engine<br>Post-instalación Linux | Linux nativo o VM Linux<br>Permisos, servicio y ajustes posteriores | <a href="#">Docker Engine</a><br><a href="#">Linux post-install</a> |
| Docker Compose                          | Aplicaciones con varios servicios                                   | <a href="#">Compose install</a>                                     |

## Regla práctica

Si una guía externa contradice la documentación oficial, se prioriza Docker Docs.

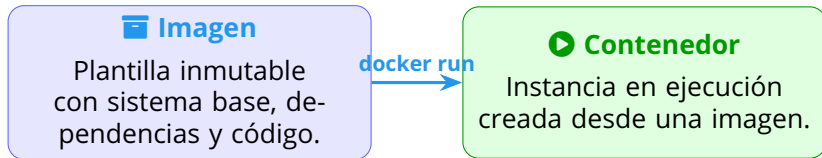


---

# Imágenes y contenedores

# Imagen vs contenedor

---

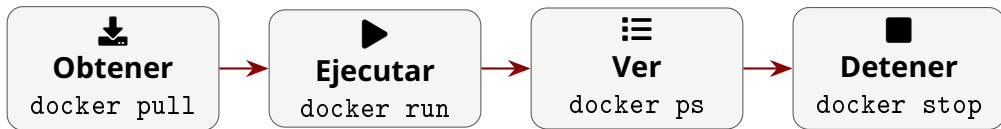


## Regla mental

La imagen es la receta; el contenedor es el plato servido y ejecutándose.

# Ciclo de vida básico

---



## Lo mínimo para operar

Ejecutar, listar, detener, eliminar contenedores y revisar imágenes locales.

---

# Comandos esenciales

# Laboratorio 1: comprobar Docker

---

```
docker --version  
docker info  
docker run hello-world
```

## Qué validamos

- La CLI responde.
- El Engine está activo.
- Docker puede descargar y ejecutar imágenes.

## Laboratorio 2: ejecutar Nginx

---

```
docker run --name web-demo -d -p 8080:80  
    nginx
```

```
docker ps
```

```
# Abrir en navegador:  
# http://localhost:8080
```

### Qué significa

- `-name`: nombre del contenedor.
- `-d`: ejecuta en segundo plano.
- `-p 8080:80`: publica el puerto.
- `nginx`: imagen usada.

## Laboratorio 3: detener y limpiar

---

```
docker stop web-demo  
docker ps -a  
docker rm web-demo  
docker images
```

### Buenas prácticas

- No acumules contenedores detenidos.
- Usa nombres claros en laboratorios.
- Verifica antes de borrar.

# Mapa rápido de comandos

---

| Comando                    | Para qué sirve                 | Ejemplo                           |
|----------------------------|--------------------------------|-----------------------------------|
| <code>docker run</code>    | Crea y ejecuta un contenedor   | <code>docker run nginx</code>     |
| <code>docker ps</code>     | Lista contenedores activos     | <code>docker ps -a</code>         |
| <code>docker stop</code>   | Detiene un contenedor          | <code>docker stop web-demo</code> |
| <code>docker rm</code>     | Elimina contenedores detenidos | <code>docker rm web-demo</code>   |
| <code>docker images</code> | Lista imágenes locales         | <code>docker images</code>        |

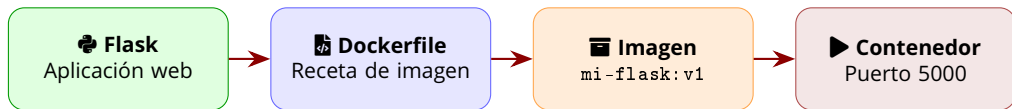


---

# Primera aplicación

# Proyecto práctico de hoy

---



## Meta

Crear una app mínima, construir su imagen y verla funcionando desde el navegador.

## Paso 1: crear archivos del proyecto

---

```
cd codigos/sesion1  
  
# Archivos ya preparados para la practica:  
# app.py  
# requirements.txt  
# Dockerfile
```

### Estructura esperada

Los archivos de la app están en `codigos/sesion1` junto al material de esta sesión.

## Paso 2: código de la app Flask

---

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return "Hola Docker desde PIT 2026"
8
9 if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
```

## Paso 3: dependencias

---

```
# requirements.txt  
flask==3.0.3
```

### Por qué declararlas

El contenedor debe poder instalar lo necesario sin depender de lo que tenga la laptop del alumno.

## Paso 4: Dockerfile mínimo

---

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9
10 EXPOSE 5000
11 CMD ["python", "app.py"]
```

## Paso 5: construir la imagen

---

```
docker build -t mi-flask:v1 .
```

```
docker images
```

### Lectura del comando

-t mi-flask:v1 asigna nombre y versión a la imagen. El punto final indica que el contexto de build es la carpeta actual.

## Paso 6: ejecutar la aplicación

---

```
docker run --name flask-demo -d -p 5000:5000 mi-flask:v1  
  
docker ps  
  
# Abrir:  
# http://localhost:5000
```

### Resultado esperado

El navegador debe mostrar: Hola Docker desde PIT 2026



## Paso 7: limpiar el laboratorio

---

```
docker stop flask-demo  
docker rm flask-demo  
  
# Opcional: borrar la imagen  
docker rmi mi-flask:v1
```

### Regla práctica

Primero detienes el contenedor, luego lo eliminas. La imagen queda disponible hasta que decidas borrarla.

# Errores frecuentes en la primera práctica

---

| Error                | Causa probable          | Solución rápida                    |
|----------------------|-------------------------|------------------------------------|
| Puerto ocupado       | Otro proceso usa 5000   | Cambiar a -p 5001:5000             |
| No abre en navegador | Contenedor detenido     | Revisar <code>docker ps -a</code>  |
| Imagen no encontrada | Nombre mal escrito      | Revisar <code>docker images</code> |
| Build falla          | Archivo faltante o typo | Verificar carpeta y Dockerfile     |

# Checklist de aprendizaje — Sesión 1

---

- Puedo explicar qué es un hipervisor y para qué sirve una VM.
- Puedo describir el flujo básico para instalar una VM de laboratorio.
- Puedo ubicar la documentación oficial para instalar Docker y Compose.
- Puedo explicar qué es la contenerización y qué problema resuelve.
- Puedo explicar por qué Docker evita diferencias entre entornos.
- Puedo diferenciar imagen y contenedor con un ejemplo.
- Puedo ejecutar, listar, detener y eliminar contenedores.
- Puedo construir una imagen propia con `docker build`.
- Puedo ejecutar mi primera app usando `docker run -p`.

# Resumen de la Sesión 1

---

- 1 Una VM virtualiza una máquina completa mediante un hipervisor.
- 2 La instalación de Docker y Compose se valida con la documentación oficial.
- 3 La contenerización empaqueta aplicaciones y dependencias.
- 4 Docker permite ejecutar entornos reproducibles.
- 5 Una imagen es una plantilla; un contenedor es una ejecución de esa imagen.
- 6 Ya ejecutamos comandos base y una primera aplicación Flask en contenedor.

## Ahora: Sesión 2

**Dockerfile Profesional:** instrucciones clave, capas, caché y buenas prácticas.

# Objetivo de la sesión 2

---

## Al terminar la sesión 2 podrás

- Escribir un Dockerfile con las instrucciones clave: FROM, RUN, COPY, ENV, EXPOSE y CMD.
- Explicar cómo funcionan las capas y cómo afectan el rendimiento del build.
- Aprovechar la caché de Docker para builds más rápidos.
- Usar .dockerignore para evitar archivos innecesarios en la imagen.
- Elegir imágenes base livianas (slim, alpine) según el caso.
- Publicar imágenes en Docker Hub con tagging y versionado.



**De Dockerfile básico a imagen profesional**

---

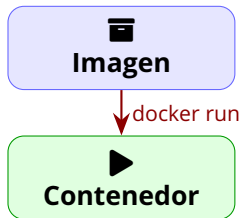
# **Recordatorio de la sesión anterior**

# Lo que ya dominamos

- Contenerización: empaquetar app + dependencias.
- Diferencia entre imagen (plantilla) y contenedor (ejecución).
- Docker resuelve el problema de «*en mi máquina funciona*».
- Comandos esenciales: `run`, `ps`, `stop`, `rm`, `images`.
- Construimos y ejecutamos una app Flask en contenedor.

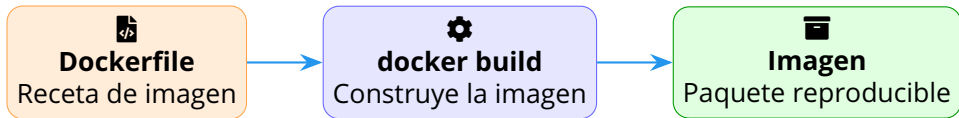
## Base para hoy

Sabemos ejecutar contenedores desde imágenes existentes. Ahora crearemos nuestras propias imágenes de forma profesional.



# ¿Qué es un Dockerfile?

---





---

# **Instrucciones clave del Dockerfile**

# FROM: el punto de partida

---

```
1 FROM python:3.12-slim
```

- Define la imagen base sobre la que se construye.
- Es la **primera instrucción** de todo Dockerfile.
- Puede ser una imagen oficial (python, node, nginx) o una imagen propia.

## Consejo

Usa imágenes oficiales siempre que puedas. Son mantenidas, actualizadas y más seguras que imágenes de terceros.

## Versiones

Evita `:latest` en producción. Fija una versión concreta (`:3.12-slim`) para builds reproducibles.

# RUN: ejecutar comandos durante el build

```
1 RUN apt-get update && \  
2     apt-get install -y curl && \  
3     rm -rf /var/lib/apt/lists/*
```

- Ejecuta comandos **durante la construcción** de la imagen.
- Se usa para instalar paquetes, crear carpetas, configurar el sistema.
- Cada RUN crea una **nueva capa** en la imagen.

## Buena práctica

Encadena comandos con `&&` para reducir capas y limpiar cachés en la misma instrucción.

## Regla de oro

Limpia archivos temporales (apt lists, pip cache) en el mismo RUN para no arrastrar basura a la imagen final.

# COPY vs ADD

```
1 # COPY: copia archivos locales
2 COPY requirements.txt .
3 COPY app.py /app/
```

- Copia archivos o carpetas desde el contexto de build (tu PC) hacia la imagen.
- Es la opción **recomendada** para la mayoría de los casos.

```
1 # ADD: copia + descomprime + URLs
2 ADD app.tar.gz /app/
3 ADD https://example.com/file /tmp/
```

- Además de copiar, descomprime automáticamente archivos `.tar.gz`.
- Puede descargar desde URLs.
- Úsalo solo cuando necesites la funcionalidad extra.

## Regla práctica

Prefiere COPY. Usa ADD solo si necesitas descomprimir automáticamente.

# WORKDIR: el directorio de trabajo

1 `WORKDIR /app`

- Establece el directorio de trabajo para las instrucciones siguientes.
- Si la carpeta no existe, Docker la **crea automáticamente**.
- Afecta a `RUN`, `CMD`, `ENTRYPOINT`, `COPY` y `ADD`.

## ¿Por qué usarlo?

- Evita rutas absolutas en cada comando.
- Hace el Dockerfile más legible.
- Previene errores de archivos en carpetas equivocadas.

## Prefiere WORKDIR

Prefiere `WORKDIR /app`; `RUN cd ...` no persiste entre instrucciones.

# ENV: variables de entorno

```
1 ENV FLASK_ENV=production
2 ENV APP_HOME=/app
3 ENV PYTHONUNBUFFERED=1
```

- Define variables de entorno disponibles en tiempo de build y en el contenedor.
- Ideal para configuraciones que no cambian entre entornos.
- Se pueden sobrescribir con `docker run -e`.

## Cuándo usar ENV

- Configuración por defecto de la app.
- Versiones de herramientas.
- Rutas y paths estándar.

## Cuándo NO

Secretos como contraseñas o tokens no deben quedar hardcodeados en la imagen.

# EXPOSE: declarar puertos

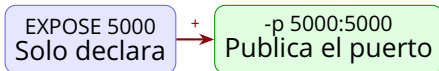
---

```
1 EXPOSE 5000
2 EXPOSE 80/tcp
```

- **Documenta** qué puertos escucha el contenedor.
- No publica el puerto automáticamente.
- Sirve como metadata para otros desarrolladores y herramientas.

## EXPOSE vs -p

- EXPOSE: declara (informativo).
- docker run -p: publica y mapea el puerto.



# CMD vs ENTRYPOINT

## CMD

```
1 CMD ["python", "app.py"]
```

- Comando por defecto al iniciar el contenedor.
- Se puede **sobrescribir** desde `docker run`.
- Forma exec (array) recomendada sobre forma shell.

## ENTRYPOINT

```
1 ENTRYPOINT ["python"]  
2 CMD ["app.py"]
```

- Define el ejecutable principal (no se sobrescribe fácilmente).
- CMD proporciona argumentos por defecto al ENTRYPOINT.
- Útil para crear herramientas CLI en contenedores.

## Regla general

Usa CMD para apps. Usa ENTRYPOINT + CMD para herramientas CLI.



# Resumen visual de instrucciones

---

| Instrucción | ¿Qué hace?                         | ¿Cuándo usarla?                         |
|-------------|------------------------------------|-----------------------------------------|
| FROM        | Define la imagen base              | Siempre, como primera línea             |
| RUN         | Ejecuta comandos en el build       | Instalar dependencias, scripts de setup |
| COPY        | Copia archivos locales a la imagen | Mover código y archivos del proyecto    |
| WORKDIR     | Establece el directorio de trabajo | Al inicio y cuando cambies de contexto  |
| ENV         | Define variables de entorno        | Configuraciones por defecto             |
| EXPOSE      | Declara puertos de escucha         | Documentar qué puertos usa la app       |
| CMD         | Comando por defecto del contenedor | Iniciar la aplicación                   |

---

# Capas, caché y buenas prácticas

# ¿Cómo construye Docker una imagen?

FROM python:3.12-slim

💎 *capa de la imagen base*

WORKDIR /app

💎 *capa 1*

COPY requirements.txt .

💎 *capa 2*

RUN pip install -r requirements.txt

💎 *capa 3*

COPY app.py .

💎 *capa 4*

CMD ["python", "app.py"]

💎 *capa 5 (metadatos)*

**Cada instrucción del Dockerfile genera una capa**

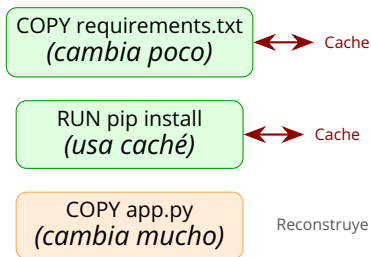
Las capas son **inmutables**. Docker reutiliza las que no cambiaron mediante caché.

# La caché de Docker: cómo funciona

- Docker compara cada instrucción con la caché de builds anteriores.
- Si la instrucción y sus dependencias no cambiaron, **reutiliza la capa**.
- Si una capa cambia, **todas las capas siguientes se reconstruyen**.

## Regla de oro del orden

Pon primero lo que **cambia menos** (instalación de dependencias) y al final lo que **cambia más** (código de la app).



# Ejemplo: Dockerfile optimizado

## Mal orden (lento)

```
1 FROM python:3.12
2
3 COPY . /app          # TODO cambia
4 WORKDIR /app
5
6 RUN pip install \     # Siempre
7     -r requirements.txt
8
9 CMD ["python", "app.py"]
```

## Buen orden (rápido)

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 # Dependencias primero
6 COPY requirements.txt .
7 RUN pip install --no-cache-dir \
8     -r requirements.txt
9
10 # Código al final
11 COPY . .
12
13 CMD ["python", "app.py"]
```

## ¿Por qué es más rápido?

Cachea dependencias; cambios en app.py no reinstalan paquetes.

# Otras buenas prácticas

## 1. Usa imágenes base específicas

```
1  # Mal: imagen generica y pesada
2  FROM python
3
4  # Bien: version fija y liviana
5  FROM python:3.12-slim
```

## 2. Limpia los paquetes no necesarios

```
1  RUN apt-get update && \
2      apt-get install -y --no-install-recommends \
3          curl && \
4      rm -rf /var/lib/apt/lists/*
```

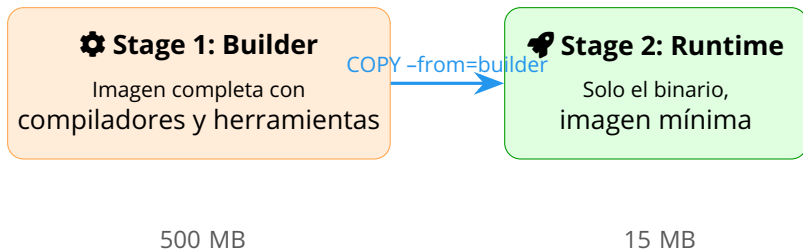
## 3. Limpia en el mismo RUN

```
1  RUN pip install -r requirements.txt && \
2      rm -rf /root/.cache/pip
```

## 4. Usa multistage builds

```
1  # Stage 1: build
2  FROM golang:1.21 AS builder
3  RUN go build -o app
4
5  # Stage 2: runtime (imagen final pequeña)
6  FROM alpine:3.19
7  COPY --from=builder /app .
8  CMD ["/app"]
```

# Multistage build: el patrón profesional



## Ventaja

La imagen final solo contiene lo necesario para ejecutar. Las herramientas de compilación quedan en la etapa de build y no contaminan la imagen de producción.

---

**.dockerignore e  
imágenes base  
livianas**



# .dockerignore: qué es y para qué sirve

```
# .dockerignore
__pycache__
*.pyc
*.pyo
.git
.env
*.log
node_modules
venv/
.venv/
.vscode/
*.md
Dockerfile
docker-compose.yml
```

- Similar a `.gitignore`: archivos y carpetas que **no** se envían al contexto de build.
- Reduce el tamaño del contexto y acelera el build.
- Evita que archivos sensibles (`.env`, claves) terminen en la imagen.

## Regla

Incluye `.dockerignore`; evita enviar `node_modules`, `.git` y credenciales.

# Imágenes base: slim vs alpine



## Completa

`python:3.12`

1 GB

Todas las herramientas  
Debian completo



## Slim

`python:3.12-slim`

150 MB

Debian mínimo  
Lo esencial para Python



## Alpine

`python:3.12-alpine`

50 MB

Alpine Linux (musl)  
La más liviana

## ¿Cuál elegir?

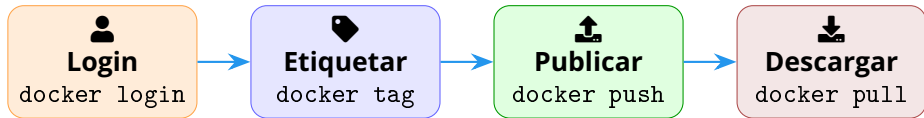
**Slim:** buena relación tamaño/compatibilidad. **Alpine:** máxima reducción, pero puede tener problemas con paquetes que dependen de glibc. Para empezar, `slim` es la opción más segura.

---

# Publicación en Docker Hub

# Flujo de publicación

---



## El ciclo completo

1. Construyes tu imagen.
2. Le asignas un tag.
3. La publicas en un registro.
4. Cualquiera puede descargarla y usarla.

## 1. Iniciar sesión en Docker Hub

```
docker login
# Usuario: tu-usuario
# Password: tu-token
```

## 2. Etiquetar la imagen

```
# Formato:
# usuario/imagen:tag

docker tag mi-flask:v1 \
  tony/mi-flask:v1
```

## Convención de tags

- v1, v2, v3: versiones específicas.
- latest: la más reciente (se actualiza manualmente).
- dev, staging, prod: por entorno.
- 2026-05-31: por fecha de build.

## Importante

latest no es automático: tú decides qué imagen lo lleva.

# Push y pull

---

## 3. Publicar la imagen

```
docker push tony/mi-flask:v1  
docker push tony/mi-flask:latest
```

La imagen queda disponible en Docker Hub según la visibilidad del repositorio.

## 4. Descargar y ejecutar

```
docker pull tony/mi-flask:v1  
  
docker run -d -p 5000:5000 \  
  tony/mi-flask:v1
```

## ¿Pública o privada?

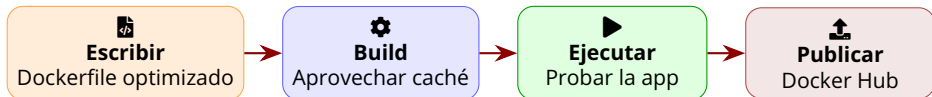
Docker Hub permite 1 repo privado gratis. GHCR es alternativa para repos privados.

---

# Laboratorio práctico

# Laboratorio: app Flask optimizada

---



## Objetivo

Partiendo del proyecto Flask de la sesión 1, optimizar el Dockerfile con buenas prácticas, construir la imagen aprovechando caché y publicarla en Docker Hub.



# Dockerfile optimizado para el laboratorio

---

```
1  # Etapa 1: builder (si necesitas compilar)
2  FROM python:3.12-slim AS builder
3  WORKDIR /app
4  COPY requirements.txt .
5  RUN pip install --no-cache-dir -r requirements.txt
6
7  # Etapa 2: imagen final (runtime)
8  FROM python:3.12-slim
9  WORKDIR /app
10 # Copiar dependencias ya instaladas desde el builder
11 COPY --from=builder /usr/local/lib/python3.12/site-packages \
12     /usr/local/lib/python3.12/site-packages
13 COPY --from=builder /usr/local/bin /usr/local/bin
14 COPY app.py .
15 EXPOSE 5000
16 CMD ["python", "app.py"]
```

# Comandos del laboratorio

---

# 1. Construir la imagen

```
docker build -t mi-flask:v2 .
```

# 2. Ver las capas generadas

```
docker history mi-flask:v2
```

# 3. Ejecutar y probar

```
docker run -d --name flask-v2 \  
-p 5000:5000 mi-flask:v2
```

```
curl http://localhost:5000
```

# 4. Login en Docker Hub

```
docker login
```

# 5. Etiquetar

```
docker tag mi-flask:v2 \  
tony/mi-flask:v2
```

# 6. Publicar

```
docker push tony/mi-flask:v2
```

# 7. Verificar en:

```
# hub.docker.com/r/tony/mi-flask
```

## Errores frecuentes en esta sesión

---

| Error                                             | Causa probable                                      | Solución rápida                                             |
|---------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------|
| Build lento siempre                               | Mal orden de capas (COPY . . antes de RUN)          | Mover dependencias antes del código                         |
| Imagen muy pesada                                 | Usar imagen base :latest completa                   | Cambiar a :slim o :alpine                                   |
| Caché no funciona                                 | Cambió un archivo antes de lo esperado              | Revisar orden: lo estable primero                           |
| docker push rechazado                             | No hay login o tag mal escrito                      | docker login y verificar usuario/imagen:tag                 |
| Archivos .env en la imagen<br>Error no space left | Falta .dockerignore<br>Imágenes acumuladas en disco | Crear .dockerignore y reconstruir<br>docker system prune -a |

## Checklist de aprendizaje — Sesión 2

---

- Puedo escribir un Dockerfile con FROM, RUN, COPY, WORKDIR, ENV, EXPOSE y CMD.
- Puedo explicar la diferencia entre COPY y ADD, CMD y ENTRYPOINT.
- Puedo explicar qué es una capa y cómo afecta el orden de las instrucciones.
- Puedo optimizar un Dockerfile para aprovechar la caché.
- Puedo usar .dockerignore para excluir archivos del contexto de build.
- Puedo elegir entre imagen base completa, slim y alpine.
- Puedo publicar una imagen en Docker Hub con tagging y versionado.
- Puedo descargar y ejecutar una imagen desde Docker Hub.

# Resumen de la Sesión 2

---

- 1 Un Dockerfile define la receta para construir imágenes propias.
- 2 FROM, RUN, COPY, ENV, EXPOSE y CMD cubren el flujo base de una app.
- 3 El orden de capas afecta directamente la velocidad del build.
- 4 .dockerignore evita enviar basura o secretos al contexto de build.
- 5 Slim y Alpine ayudan a reducir tamaño, con criterios distintos.
- 6 Docker Hub permite compartir imágenes mediante tags versionados.

## Ahora: Sesión 3

**Docker Compose:** levantar aplicaciones multi-contenedor con Flask, PostgreSQL y variables de entorno.

# Objetivo de la sesión 3

---

## Al terminar la sesión 3 podrás

- Explicar para qué sirve Docker Compose.
- Levantar proyectos reales con múltiples servicios.
- Entender la estructura de `docker-compose.yml`.
- Conectar una aplicación Flask con PostgreSQL.
- Configurar variables de entorno usando archivo `.env`.
- Ejecutar, revisar y detener un stack completo con Compose.



**Un comando, varios servicios**

---

# Docker Compose

# El problema: una app ya no vive sola

---

- Una app real suele necesitar base de datos, cache, proxy o workers.
- Ejecutar cada contenedor a mano vuelve el laboratorio repetitivo.
- Los comandos `docker run` crecen con puertos, redes, volúmenes y variables.
- El equipo necesita una forma reproducible de levantar todo el stack.



## Pregunta clave

¿Cómo describimos una aplicación completa, no solo un contenedor?



# Qué es Docker Compose

---



## Definición práctica

Docker Compose permite definir y ejecutar aplicaciones multi-contenedor usando un archivo YAML versionable.

# De docker run a docker compose

---

## Sin Compose

```
docker network create app-net
docker run -d --name db \
  --network app-net postgres:16
docker run -d --name web \
  --network app-net -p 5000:5000 \
  mi-flask:v3
```

## Con Compose

```
docker compose up -d
docker compose ps
docker compose down
```

El archivo `docker-compose.yml` guarda la arquitectura del proyecto.

---

# Estructura de compose.yml

# Piezas principales de Compose

---



**services**  
Contenedores  
del proyecto



**networks**  
Comunicación interna



**volumes**  
Datos persistentes



**depends\_on**  
Orden de arranque

## Idea mental

Compose describe la topología local: qué servicios existen, cómo se conectan y qué datos conservan.

# docker-compose.yml mínimo

---

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
```

## Lectura rápida

web construye la app. db usa PostgreSQL. Compose crea una red interna por defecto.

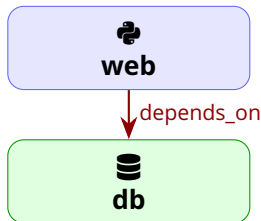
# Comandos esenciales de Compose

---

| Comando                                 | Para qué sirve                     | Uso típico          |
|-----------------------------------------|------------------------------------|---------------------|
| <code>docker compose up -d</code>       | Levanta el stack en segundo plano  | Iniciar laboratorio |
| <code>docker compose ps</code>          | Lista servicios del proyecto       | Ver estado          |
| <code>docker compose logs -f</code>     | Muestra logs integrados            | Diagnóstico         |
| <code>docker compose exec web sh</code> | Abre shell en un servicio          | Debug               |
| <code>docker compose down</code>        | Detiene y elimina contenedores/red | Limpiar stack       |

# Dependencias entre servicios

- `depends_on` define orden de arranque básico.
- No garantiza que la base de datos ya acepte conexiones.
- Para producción o laboratorios robustos se usan healthchecks.
- En esta sesión nos enfocamos en levantar y conectar el stack.



## Regla práctica

Arrancar antes no significa estar listo para recibir tráfico.

---

# Flask + PostgreSQL



# Arquitectura del laboratorio

---



## Clave de red interna

Dentro de Compose, la app no se conecta a `localhost`; se conecta al nombre del servicio: `db`.

# Variables de conexión en Flask

---

```
1 import os
2
3 DB_HOST = os.getenv("DB_HOST", "db")
4 DB_NAME = os.getenv("DB_NAME", "
    appdb")
5 DB_USER = os.getenv("DB_USER", "
    appuser")
6 DB_PASSWORD = os.getenv("DB_PASSWORD
    ")
```

## Por qué usar variables

- Evitan hardcodear credenciales.
- Permiten cambiar entorno sin tocar código.
- Funcionan bien con `.env` y Compose.

# Compose para Flask + PostgreSQL

---

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    env_file:
      - .env
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

---

# Variables de entorno

# Archivo .env del proyecto

---

```
DB_HOST=db
DB_NAME=appdb
DB_USER=appuser
DB_PASSWORD=appsecret
FLASK_ENV=development
```

## Importante

No subas `.env` con credenciales reales a GitHub. Usa `.env.example` para documentar variables esperadas.

## Para clase

Usaremos valores simples de laboratorio para entender el flujo.

# .env vs environment

| Opción                    | Ventaja                | Cuándo usarla                |
|---------------------------|------------------------|------------------------------|
| <code>environment</code>  | Visible en el YAML     | Valores simples o explícitos |
| <code>env_file</code>     | Separa configuración   | Variables por entorno        |
| <code>.env.example</code> | Documenta sin secretos | Repos compartidos            |
| Secret manager            | Mejor seguridad        | Producción real              |

## Regla práctica

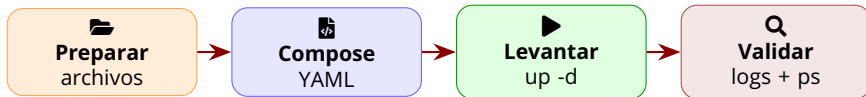
El archivo `.env` ayuda en desarrollo; no es una bóveda de secretos.

---

# **Laboratorio multi-contenedor**

# Flujo del laboratorio

---



## Meta

Ejecutar una aplicación Flask conectada a PostgreSQL usando un solo archivo Compose.



# Comandos del laboratorio Compose

---

```
# 1. Levantar el stack  
docker compose up -d --build
```

```
# 2. Revisar servicios  
docker compose ps
```

```
# 3. Ver logs integrados  
docker compose logs -f
```

```
# 4. Probar la app  
curl http://localhost:5000
```

```
# 5. Entrar al servicio web  
docker compose exec web sh
```

```
# 6. Detener y limpiar  
docker compose down
```

# Errores frecuentes en Compose

---

| Error               | Causa probable                                       | Solución rápida                        |
|---------------------|------------------------------------------------------|----------------------------------------|
| App no conecta a BD | Usa <code>localhost</code> en vez de <code>db</code> | Cambiar host al nombre del servicio    |
| Puerto ocupado      | Otro proceso usa 5000                                | Cambiar a <code>5001:5000</code>       |
| Variables vacías    | Falta <code>.env</code> o nombre incorrecto          | Revisar <code>env_file</code> y claves |
| BD pierde datos     | No hay volumen persistente                           | Agregar volumen nombrado               |
| Cambios no aparecen | Imagen antigua en caché                              | Usar <code>up -d --build</code>        |

## Checklist de aprendizaje — Sesión 3

---

- Puedo explicar para qué sirve Docker Compose.
- Puedo leer la estructura básica de `docker-compose.yml`.
- Puedo definir servicios, puertos, dependencias y volúmenes.
- Puedo conectar Flask con PostgreSQL usando el nombre del servicio.
- Puedo configurar variables con `.env` y `env_file`.
- Puedo levantar y detener un stack con `docker compose up` y `down`.

# Resumen de la Sesión 3

---

- 1 Compose describe aplicaciones multi-contenedor en un archivo YAML.
- 2 Los servicios se comunican por nombre dentro de la red del proyecto.
- 3 `ports` publica hacia la máquina host; la red interna usa nombres de servicio.
- 4 `volumes` permite conservar datos de PostgreSQL entre reinicios.
- 5 `.env` separa configuración del código y del YAML.

## Próxima sesión

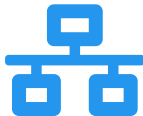
**Redes, volúmenes y persistencia:** comunicación interna, almacenamiento y datos durables.

# Objetivo de la sesión 4

---

## Al terminar la sesión 4 podrás

- Explicar cómo se comunican contenedores mediante redes Docker.
- Diferenciar red interna, puertos publicados y nombres de servicio.
- Usar volúmenes nombrados y bind mounts para persistencia.
- Aislar servicios internos como bases de datos.
- Agregar healthchecks para validar disponibilidad.
- Realizar backup y restauración básica de datos del proyecto.



**Datos durables y redes  
claras**

---

# Redes Docker

# Qué problema resuelven las redes

---

- Los contenedores necesitan comunicarse sin depender de IPs manuales.
- Compose crea una red interna por proyecto automáticamente.
- Cada servicio puede resolverse por su nombre: `web`, `db`, `cache`.
- Solo publicamos hacia el host lo que debe ser accesible desde fuera.



## Idea clave

Dentro de la red Docker, el nombre del servicio funciona como DNS interno.

# Red interna vs puerto publicado

---



## Regla práctica

Publica la app web si necesitas navegador. No publiques la base de datos si solo la consume otro contenedor.



# Definir una red explícita en Compose

---

```
services:
  web:
    build: .
    networks:
      - backend

  db:
    image: postgres:16
    networks:
      - backend

networks:
  backend:
```

## Cuándo hacerlo

Define redes explícitas para separar tráfico o documentar mejor la arquitectura.

# Comandos útiles para redes

---

| Comando                                         | Uso                                           |
|-------------------------------------------------|-----------------------------------------------|
| <code>docker network ls</code>                  | Lista redes existentes                        |
| <code>docker network inspect &lt;red&gt;</code> | Muestra contenedores conectados               |
| <code>docker compose ps</code>                  | Relaciona servicios y puertos publicados      |
| <code>docker compose exec web sh</code>         | Permite probar conectividad desde el servicio |

## Prueba rápida

Desde `web`, el host de PostgreSQL debe ser `db`, no `localhost`.

---

# Volúmenes y persistencia

# Por qué se pierden datos

---

- Un contenedor es reemplazable: puede eliminarse y recrearse.
- Los datos escritos dentro del filesystem del contenedor no son una estrategia durable.
- Las bases de datos necesitan almacenamiento externo al ciclo de vida del contenedor.
- Docker ofrece volúmenes y bind mounts para resolverlo.

## Regla mental

Contenedor reemplazable; datos persistentes.



**Datos fuera del contenedor**

# Volumen nombrado vs bind mount

| Tipo             | Qué es                                 | Cuándo usarlo                  |
|------------------|----------------------------------------|--------------------------------|
| Volumen nombrado | Almacenamiento administrado por Docker | Datos de bases de datos        |
| Bind mount       | Carpeta local montada en el contenedor | Desarrollo y edición de código |
| Sin volumen      | Datos dentro del contenedor            | Solo pruebas descartables      |

## Analogía

El contenedor es una laptop prestada; el volumen es el disco externo donde guardas lo importante.

# Volumen nombrado para PostgreSQL

---

```
services:
  db:
    image: postgres:16
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

## Clave

Si eliminas el contenedor, el volumen puede seguir existiendo y conservar los datos.

# Bind mount para desarrollo

---

```
services:
  web:
    build: .
    volumes:
      - ./app:/app
    ports:
      - "5000:5000"
```

## Uso típico

Montar código local dentro del contenedor para desarrollo. Para producción, normalmente se prefiere copiar el código en la imagen.

# Comandos útiles para volúmenes

---

| Comando                                                            | Uso                                          |
|--------------------------------------------------------------------|----------------------------------------------|
| <code>docker volume ls</code>                                      | Lista volúmenes                              |
| <code>docker volume inspect</code><br><code>&lt;volumen&gt;</code> | Muestra detalles del volumen                 |
| <code>docker compose down</code>                                   | Borra contenedores y red, conserva volúmenes |
| <code>docker compose down -v</code>                                | Borra también los volúmenes del proyecto     |

## Cuidado

`down -v` elimina datos persistentes del stack. Úsalo solo si realmente quieres reiniciar desde cero.



---

# **Servicios internos**

# Qué servicios deben ser internos

- La app web suele publicar un puerto para el navegador.
- La base de datos normalmente solo debe recibir tráfico desde la app.
- Redis, workers y colas suelen ser servicios internos.
- Menos puertos publicados significa menor superficie de exposición.



## Regla práctica

Si el usuario no necesita entrar directamente, no publiques el puerto.

# Base de datos interna en Compose

---

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: postgres:16
    # Sin ports: no se expone al host
```

## Lectura

web está publicado para el navegador. db solo está disponible dentro de la red Docker del proyecto.

---

# Healthchecks y backup

# Por qué usar healthchecks

---

- Un contenedor puede estar encendido pero no listo.
- PostgreSQL puede tardar segundos en aceptar conexiones.
- Un healthcheck permite declarar cómo verificar el estado real.
- Mejora la observabilidad durante laboratorios y despliegues.

## Idea clave

`running` no siempre significa `healthy`.



**Listo para operar**

# Healthcheck para PostgreSQL

---

```
services:
  db:
    image: postgres:16
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
```

## Lectura

Docker ejecuta `pg_isready`. Si responde correctamente, marca el servicio como saludable.

# Backup básico de PostgreSQL

---

```
# Crear carpeta local para backups
mkdir backups

# Exportar datos desde el servicio db
docker compose exec -T db pg_dump \
-U appuser appdb > backups/appdb.sql
```

## Regla práctica

Un volumen conserva datos localmente, pero backup significa tener una copia exportable fuera del contenedor.

# Restaurar información

---

```
# Restaurar desde el archivo SQL
docker compose exec -T db psql \
  -U appuser appdb < backups/appdb.sql
```

## Antes de restaurar

Verifica que el servicio db esté arriba, que el usuario sea correcto y que el archivo exista en tu máquina.

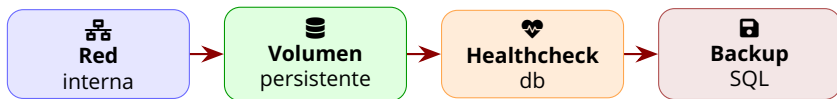


---

# Laboratorio datos y redes

# Flujo del laboratorio

---



## Meta

Mejorar el stack Flask + PostgreSQL para que la base de datos sea interna, persistente y respaldable.

# Comandos del laboratorio

---

# 1. Levantar stack

```
docker compose up -d --build
```

# 2. Revisar puertos

```
docker compose ps
```

# 3. Inspeccionar red

```
docker network ls
```

```
docker network inspect <red>
```

# 4. Ver volumen

```
docker volume ls
```

# 5. Backup

```
mkdir backups
```

```
docker compose exec -T db pg_dump \  
-U appuser appdb > backups/appdb.  
sql
```

# Errores frecuentes en datos y redes

---

| Error                  | Causa probable                     | Solución rápida                                     |
|------------------------|------------------------------------|-----------------------------------------------------|
| App no conecta a BD    | Host incorrecto o red distinta     | Usar db y misma red                                 |
| BD expuesta al host    | Se agregó <code>ports</code> en db | Quitar <code>ports</code> si no es necesario        |
| Datos desaparecen      | Sin volumen persistente            | Agregar volumen nombrado                            |
| Backup falla           | Usuario o BD incorrectos           | Revisar <code>DB_USER</code> y <code>DB_NAME</code> |
| Servicio no está listo | Falta healthcheck o espera         | Revisar logs y estado healthy                       |

## Checklist de aprendizaje — Sesión 4

---

- Puedo explicar cómo Docker resuelve nombres de servicios en una red.
- Puedo diferenciar puerto publicado y comunicación interna.
- Puedo usar volúmenes nombrados para persistir datos.
- Puedo usar bind mounts para desarrollo local.
- Puedo mantener PostgreSQL como servicio interno.
- Puedo crear un backup y restaurarlo con comandos básicos.

## Resumen de la Sesión 4

---

- 1 Los servicios se comunican por nombre dentro de redes Docker.
- 2 Los puertos publicados son para acceso desde el host, no para comunicación interna.
- 3 Los volúmenes nombrados conservan datos aunque se recree el contenedor.
- 4 Los bind mounts son útiles para desarrollo y edición local.
- 5 Los healthchecks verifican disponibilidad real, no solo ejecución.
- 6 Backup y restauración convierten datos persistentes en datos recuperables.

### Próxima sesión

**Docker en producción:** Nginx, multi-stage builds, credenciales seguras, logging y monitoreo.

# Resumen del programa (hasta ahora)

---

- 1** Docker resuelve el problema de reproducibilidad entre entornos.
- 2** Contenerizar es empaquetar app + dependencias para ejecución uniforme.
- 3** Una VM virtualiza una máquina completa; un contenedor comparte el kernel del host.
- 4** La instalación de Docker se guía por la documentación oficial, no de memoria.
- 5** Imagen = plantilla inmutable; contenedor = instancia en ejecución.
- 6** Comandos esenciales: run, ps, stop, rm, images, build.
- 7** Un Dockerfile es la receta profesional para construir imágenes propias.
- 8** Las capas son inmutables: el orden correcto acelera los builds con caché.
- 9** Slim y Alpine reducen drásticamente el tamaño de las imágenes.
- 10** Docker Hub permite compartir imágenes con tagging y versionado.
- 11** Docker Compose permite levantar stacks multi-contenedor reproducibles.
- 12** Flask y PostgreSQL pueden comunicarse usando nombres de servicio.
- 13** Redes Docker separan comunicación interna de puertos publicados.
- 14** Volúmenes, healthchecks y backups hacen el stack más durable.

## Próxima sesión

**Docker en producción:** reverse proxy, credenciales, logging y monitoreo.